

Code Changes Explained

Every fix applied during the Postgres → Supabase migration,
with the error, the cause, and the change.

1 docker-compose.yml -- switch from local Postgres to Supabase

What we were doing: Moving the database from a local `pgvector/pgvector` Docker container to a hosted Supabase Postgres instance.

The change: Removed the `db` service and `postgres_data` volume entirely. The backend's `DATABASE_URL` now comes from an environment variable instead of a hardcoded local connection string, and `SUPABASE_URL`, `LANGCHAIN_API_KEY_GEMINI`, `LANGCHAIN_API_KEY_GEMINI_2` were added so the container actually receives them (see #6 below for why this mattered).

```
services:
  redis:
    image: redis:7-alpine
    container_name: contentcoach-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data

  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: contentcoach-backend
    restart: unless-stopped
    environment:
      ENV: development
      DATABASE_URL: ${DATABASE_URL}
      REDIS_URL: redis://redis:6379
      SUPABASE_URL: ${SUPABASE_URL}
      LANGCHAIN_API_KEY_GEMINI: ${LANGCHAIN_API_KEY_GEMINI}
      LANGCHAIN_API_KEY_GEMINI_2: ${LANGCHAIN_API_KEY_GEMINI_2}
    ports:
      - "8000:8000"
    depends_on:
      redis:
        condition: service_started

volumes:
  redis_data:
```

2 Root `.env` -- point at the real Supabase connection string

Error we hit:

```
sqlalchemy.exc.NoSuchModuleError: Can't load plugin: sqlalchemy.dialects:https
```

Why it happened: `DATABASE_URL` had been set to Supabase's **Project URL** (`https://<ref>.supabase.co` -- used for the REST/Auth/Storage APIs), not an actual Postgres connection string. SQLAlchemy tried to parse `https` as the database dialect/driver name and failed, since `https` isn't a Postgres driver.

The fix: Replaced it with the real Postgres URI from the Supabase dashboard (Connect -> Direct -> connection string).

```
DATABASE_URL=postgresql://postgres.<ref>:[PASSWORD]@aws-0-<region>.pooler.supabase.com:5432/postgres
SUPABASE_URL=https://<ref>.supabase.co
LANGCHAIN_API_KEY_GEMINI=<key>
LANGCHAIN_API_KEY_GEMINI_2=
```

A second error surfaced right after fixing the first one:

```
psycopg2.OperationalError: connection to server at "db.<ref>.supabase.co"
(2600:1f1c:...) port 5432 failed: Network is unreachable
```

Why: Supabase's *direct* connection host resolves to an **IPv6-only** address. Free-tier Supabase doesn't include the IPv4 add-on, and most local/home networks plus Docker's default networking are IPv4-only -- so the container could never route to that address.

The fix: Switched to the **Session pooler** connection string instead (the `aws-0-<region>.pooler.supabase.com` host above), which is IPv4-compatible on every Supabase tier. Session mode specifically (not Transaction mode / port 6543) was required because Alembic migrations need prepared-statement/DDL support that transaction-mode pooling doesn't allow.

3 frontend/.env -- give Vite the real Supabase values

Error we hit: Signup failed in the browser with `Failed to fetch`, and DevTools showed requests going to `placeholder.supabase.co` with `ERR_NAME_NOT_RESOLVED`.

Why it happened: `frontend/src/lib/supabaseClient.js` had a fallback built in for exactly this situation:

```
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY || 'placeholder-anon-key';
```

`VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` were never actually reaching Vite. The real values *did* exist in a `.env` file -- but at the **repo root**, and Vite only auto-loads `.env` files from the **frontend project root** (next to `package.json` / `vite.config.js`). So Vite silently fell back to the placeholder every time.

The fix: Created `frontend/.env` with the Vite-prefixed values, and restarted the dev server (Vite only reads env vars at startup, not on hot reload).

```
VITE_SUPABASE_URL=https://<ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<anon/publishable key>
```

4 backend/core/config.py + docker-compose.yml -- add SUPABASE_URL

Error we hit: Every single backend API route started returning `401 Unauthorized`, even for a freshly signed-up, logged-in user.

Why it happened: `backend/auth/service.py` builds the JWKS URL (used to verify Supabase JWTs) like this:

```
jwks_url = f"{settings.SUPABASE_URL}/auth/v1/.well-known/jwks.json"
```

and `settings.SUPABASE_URL` defaults to `""` in `config.py`. With no value set anywhere, the URL resolved to just `/auth/v1/.well-known/jwks.json` -- not a real endpoint -- so **every** token verification failed, regardless of whether the token itself was valid.

The fix: Added `SUPABASE_URL` to the root `.env` (see #2 above) *and* explicitly listed it under the backend service's `environment:` block in `docker-compose.yml` (see #1 above). That second part turned out to be the actual missing piece -- see the note in #6 below about why this class of bug kept recurring.

5 backend/auth/service.py -- fix a duplicate-user race condition

Error we hit:

```
psycopg2.errors.UniqueViolation: duplicate key value violates unique
constraint "users_pkey"
DETAIL: Key (id)=(...) already exists.
```

This surfaced as `500 Internal Server Error` on `/api/linkedin/connection-status` and `/api/ai/sessions`.

Why it happened: On page load, the frontend fires several requests in parallel (`/api/profile`, `/api/linkedin/connection-status`, `/api/ai/sessions`, etc.). Each one independently runs `UserSyncService.get_or_create()`, which checks "does this user row exist?" before inserting one if not. Since several requests check at nearly the same instant, more than one sees "no" and tries to `INSERT` the same user -- the second one to commit hits the unique-key constraint and crashes.

The fix: Catch the race instead of letting it crash -- if the insert fails because another concurrent request already created the row, roll back and just read that row instead.

```
from sqlalchemy.exc import IntegrityError

class UserSyncService:
    @staticmethod
    def get_or_create(db: Session, identity: AuthenticatedUser) -> User:
        user_id = UUID(identity.id)
        user = db.get(User, user_id)
        if user:
            return user
```

```

logger.info("Provisioning shadow user row: user_id=%s", user_id)
user = User(id=user_id, email=identity.email, username=identity.username)
db.add(user)
try:
    db.commit()
except IntegrityError:
    # Concurrent request already inserted this user between our
    # get() check and this commit -- fall back to reading it.
    db.rollback()
    user = db.get(User, user_id)
    if user is None:
        raise
    return user
db.refresh(user)
return user

```

6 frontend/src/api/ai.js -- fix streamQuery's auth header

Error we hit: Every endpoint worked except `/api/ai/stream`, which kept returning `401 Unauthorized` even after login worked fine everywhere else.

Why it happened: Every other API call in the app goes through an axios instance with `attachAuthHeader()` wired in as an interceptor -- it reads the current Supabase session and attaches `Authorization: Bearer <token>` automatically. But `streamQuery()` uses a raw `fetch()` call instead of axios (needed for reading a streamed response body), so it never goes through that interceptor. It was still sending the **old pre-Supabase auth header** left over from before the migration:

```

const uid = localStorage.getItem('user_id') || '';
...
headers: { 'Content-Type': 'application/json', 'X-User-Id': uid }

```

The backend's `SupabaseAuth` dependency expects a Bearer JWT -- it never got one, so it always rejected the request.

The fix: Manually replicate `attachAuthHeader`'s logic inline, since raw `fetch` can't use axios interceptors.

```

import { supabase } from '../lib/supabaseClient';

const IS_LOCAL_AUTH = import.meta.env.VITE_AUTH_MODE === 'local';

export function streamQuery(prompt, sessionId, onToken, onDone, onError, onActivity) {
  const controller = new AbortController();
  (async () => {
    try {
      const headers = { 'Content-Type': 'application/json' };

      if (IS_LOCAL_AUTH) {
        const userId = localStorage.getItem('user_id');
        if (userId) headers['X-User-Id'] = userId;
      } else {
        const { data } = await supabase.auth.getSession();
        if (data.session) {
          headers['Authorization'] = `Bearer ${data.session.access_token}`;
        }
      }

      const response = await fetch('/api/ai/stream', {
        method: 'POST',
        headers,
        body: JSON.stringify({ prompt, session_id: sessionId }),
        signal: controller.signal,
      });
      if (!response.ok) {
        onError(`Server error ${response.status}`);
        return;
      }
      // ...rest of the reader/decoder loop unchanged
    }
  })();
}

```

7 backend/ai/checkpointing/factory.py + backend/main.py -- connection pool

Error we hit:

```
psycopg.OperationalError: sending prepared query failed: another command is
already in progress
```

This showed up as **500 Internal Server Error** on `GET /api/ai/sessions/{id}/threads`, specifically when switching between chat sessions quickly (i.e. under concurrent load).

Why it happened: In `backend/main.py`, the checkpointer and store were created **once** at app startup, inside the FastAPI `lifespan` context manager, and reused as shared singletons for the entire lifetime of the running server:

```
async with create_checkpointer(settings.DATABASE_URL) as checkpointer, \
    create_store(settings.DATABASE_URL) as store:
```

`AsyncPostgresSaver.from_conn_string(...)` (what `create_checkpointer` was calling under the hood) opens **one single raw psycopg connection** by default -- not a pool. Psycopg's async connections are not safe to use from more than one coroutine at the same time. So the moment two requests hit the checkpointer concurrently (e.g. two `/threads` calls fired back-to-back), the second one collided with the first mid-query and got this error.

The fix: Open a proper connection **pool** once at startup, and construct the checkpointer/store from that pool instead of from a single connection string. Now each concurrent request gets its own connection pulled from the pool (`max_size=20`), instead of every request fighting over one shared connection.

`backend/ai/checkpointing/factory.py`:

```
from contextlib import AbstractAsyncContextManager
from psycopg_pool import AsyncConnectionPool
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
from langgraph.store.postgres import AsyncPostgresStore
from backend.core.config import settings

_SESSION_TTL_MINUTES = 60 * 24 * 7

_session_title_embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001",
    task_type="retrieval_document",
    output_dimensionality=768,
    google_api_key=settings.LANGCHAIN_API_KEY_GEMINI,
)

def create_pool(database_url: str) -> AsyncConnectionPool:
    """Shared connection pool -- each concurrent request gets its own
    connection instead of every request fighting over one shared
    psycopg connection."""
    return AsyncConnectionPool(
        conninfo=database_url,
        max_size=20,
        kwargs={"autocommit": True, "prepare_threshold": 0},
        open=False,
    )

def create_checkpointer(pool: AsyncConnectionPool) -> AsyncPostgresSaver:
    """Single seam for the checkpointer backend -- swap implementations here only."""
    return AsyncPostgresSaver(pool)

def create_store(pool: AsyncConnectionPool) -> AsyncPostgresStore:
    """Single seam for the long-term (cross-thread/cross-session) store backend.
    TTL sweeper must be started explicitly (store.start_ttl_sweeper()) after
    store.setup() -- this factory only constructs the configured instance.
    """
    return AsyncPostgresStore(
        pool,
        ttl={
            "default_ttl": _SESSION_TTL_MINUTES,
            "refresh_on_read": True,
            "sweep_interval_minutes": 60,
        },
        index={
            "dims": 768,
            "embed": _session_title_embeddings,
            "fields": ["title"],
        },
    )
```

`backend/main.py`:

```

from backend.ai.checkpointing.factory import create_pool, create_checkpointer, create_store

@asynccontextmanager
async def lifespan(app: FastAPI):
    pool = create_pool(settings.DATABASE_URL)
    async with pool:
        checkpointer = create_checkpointer(pool)
        store = create_store(pool)
        await checkpointer.setup() # idempotent -- creates checkpoints/checkpoint_blobs/checkpoint_writes
        await store.setup() # idempotent -- creates store/store_migrations + vector tables (chat_sessions.title index)
        await store.start_ttl_sweeper() # sweeps expired chat_sessions rows (7-day TTL) every 60 min
        app.state.assistant = build_assistant(checkpointer, store)
        app.state.store = store
        set_assistant(app.state.assistant) # lets tools.py's get_session_context call aget_state() without a circular import
        logger.info("LangGraph assistant compiled with AsyncPostgresSaver checkpointer + AsyncPostgresStore (pooled, max_size=20)")
        try:
            yield
        finally:
            await store.stop_ttl_sweeper()

```

Recurring pattern across all of this

Two root causes explain most of the errors above:

1. **An env var existed in `.env` but was never listed under the backend service's `environment:` block in `docker-compose.yml`.** Compose's `.env` file only fills `${VAR}` placeholders *inside* `docker-compose.yml` itself -- it does not automatically forward every `.env` variable into the running container. This caused both the `SUPABASE_URL` 401s (#4) and the earlier `LANGCHAIN_API_KEY_GEMINI` embedding failures.
2. **Vite only auto-loads `.env` from the frontend project root**, not the repo root -- causing the `placeholder.supabase.co` signup failure (#3).

Worth checking both of these first on any future "why isn't this value reaching X" issue, before assuming the value itself is wrong.